table_generation

October 22, 2026

1 Sign Table: Structure and Mechanics

The `clif` package uses a *virtual* sign table to compute the geometric product. Rather than storing a full $2^d \times 2^d$ table of signs, it uses a pair of **key arrays** (row keys and column keys) from which any sign can be reconstructed on the fly. Memory cost: 2×2^d integers instead of 4^d bytes.

1.1 Ordinal (XOR-basis) indexing

Each basis blade is identified by a non-negative integer whose binary representation names the basis vectors it contains. For a 3D algebra:

Ordinal	Binary	Blade	Grade
0	000	scalar	0
1	001	e_1	1
2	010	e_2	1
3	011	e_1e_2	2
4	100	e_3	1
5	101	e_1e_3	2
6	110	e_2e_3	2
7	111	$e_1e_2e_3$	3

The result blade ordinal of $e_I \cdot e_J$ is always `I XOR J`. The sign ± 1 depends on the parity of permutations needed to sort the combined index.

[1]:
```
import clif.context as ctx
from clif.context import Cl
from clif.multivector import Accum
import numpy as np
```

1.2 Grade table

[2]:
```
Cl(3)
print("Ordinal  binary   grade")
for i in range(2**3):
    print(f"  {i:3d}    {i:03b}    {ctx.Grade[i]}")
```

```
Ordinal  binary   grade
    0    000    0
```

```
1    001    1
2    010    1
3    011    2
4    100    1
5    101    2
6    110    2
7    111    3
```

1.3 Virtual sign table structure

The `SignTable` object stores two key arrays. The sign of $e_I \cdot e_J$ is computed from `row_keys[I]` and `col_keys[J]` without materialising the full matrix.

[3]:
```
Cl(3)
table = ctx._ActiveTable
print(f"Algebra dimension:  {table.dim}")
print(f"Basis size (2^d):   {table.size}")
print(f"row_keys: {table.row_keys}")
print(f"col_keys: {table.col_keys}")
print(f"Memory: {2 * table.size} integers  (vs {table.size**2} for a full␣
 ↪table)")
```

```
Algebra dimension:  3
Basis size (2^d):   8
row_keys: [0 6 4 2 0 6 4 2]
col_keys: [0 0 1 1 3 3 2 2]
Memory: 16 integers  (vs 64 for a full table)
```

1.4 Geometric product via fast_mul

`fast_mul` is a Numba-jitted function that takes two coefficient arrays and returns the product coefficient array using the virtual sign table.

[4]:
```
mul = table.fast_mul

def basis_vec(index, size):
    v = np.zeros(size); v[index] = 1.0; return v

n = table.size

e1 = basis_vec(1, n)   # ordinal 1 = 0b001 = e1
e2 = basis_vec(2, n)   # ordinal 2 = 0b010 = e2
e3 = basis_vec(4, n)   # ordinal 4 = 0b100 = e3

# e1 * e2 → blade e1^e2 (ordinal 3 = 1 XOR 2)
r = mul(e1, e2)
print(f"e1 * e2: ordinal {np.argmax(np.abs(r))}, sign {r[3]:+.0f}")

# e2 * e1 → same blade, opposite sign
```

```
r = mul(e2, e1)
print(f"e2 * e1: ordinal {np.argmax(np.abs(r))}, sign {r[3]:+.0f}")

# e1 * e1 → scalar (Euclidean: e1^2 = +1)
r = mul(e1, e1)
print(f"e1 * e1: scalar = {r[0]:+.0f}")
```

```
e1 * e2: ordinal 3, sign -1
e2 * e1: ordinal 3, sign +1
e1 * e1: scalar = +1
```

1.5 Multivector product verification

Using `Accum` arithmetic (which calls `fast_mul` internally) to verify that $A \cdot A^{-1} = e_0$.

[5]:
```
from clif.inverse import fls_inverse

Cl(6)
A = Accum.random()
iA = fls_inverse(A)
check = A * iA
print(f"A * A^-1 scalar component: {check.Reg[0]:.15f}")
print(f"Max off-scalar residual:   {max(abs(check.Reg[1:])):.2e}")
```

```
A * A^-1 scalar component: 1.000000000000000
Max off-scalar residual:   2.05e-13
```

1.6 Memory cost vs dimension

The virtual table stays small even at large d; a full materialised table would be prohibitively large.

[6]:
```
print(f"{'d':>4}  {'2^d':>8}  {'virtual (ints)':>15}  {'full table (MB)':>16}")
print("-" * 52)
for d in range(3, 14):
    n = 2**d
    virtual = 2 * n
    full_mb = n * n / (1024**2)   # 1 byte per entry
    print(f"  {d:2d}  {n:8,}  {virtual:15,}  {full_mb:16.2f}")
```

```
   d       2^d   virtual (ints)   full table (MB)
----------------------------------------------------
   3         8               16             0.00
   4        16               32             0.00
   5        32               64             0.00
   6        64              128             0.00
   7       128              256             0.02
   8       256              512             0.06
   9       512            1,024             0.25
  10     1,024            2,048             1.00
```

11	2,048	4,096	4.00
12	4,096	8,192	16.00
13	8,192	16,384	64.00